In listing 5.5 you notice that passing all of the arguments for a constructor through to the `super` constructor is repetitive. CoffeeScript also provides `super` as a keyword for this situation:

```
class Gallery

class Camera extends Product
  constructor: (name, info) ->
    @gallery = new Gallery
    super

pixelmatic = new Camera 'The Pixelmatic 5000', {}
pixelmatic.name
# 'The Pixelmatic 5000'
```

Now you know that the CoffeeScript class sugar has been hiding things from you and that syntactic sugar is a language convenience that makes some common expressions easier. When you need to break away from classes, you'll have to dive back down into objects and prototypes, leaving the sugar behind. So, when do you need to break away from classes?

5.5. Modifying prototypes

Remember, there are plain, old objects and prototype links underneath your classes. In this section you'll learn how CoffeeScript's class declarations work by seeing the actual JavaScript they compile to. You'll also learn what you can do to classes and their instances when you start dynamically modifying their prototypes.

5.5.1. How class declarations work

Imagine now that Agtron has extra stock of some products, and he wants to offer special deals on them. If you know beforehand that Agtron is offering special deals on `Cameras` with the name `'Lacia'`, then you can handle the special deal in the class declaration: